

PROJECT

팀 프로젝트 발표

세계 축구선수(공격수)들의 몸값과 연관된 데이터 찾기

강남대학교 인공지능융합공학부

202404222 박시현

202404252 원종우

목차 LIST

- 01 프로젝트 개요
- 02 팀 구성 및 역할
- 03 프로젝트 진행과정
- 04 평가

01 프로젝트 개요 PROJECT

- 선정 주제

세계 축구선수들의 몸값과 연관된 데이터 찾기

- 연구 기간

2025.04.02 ~05.24

- 목표

축구선수의 실제 몸값과 관련된 데이터를 수집,
선수 가치에 영향을 주는 Top 10 요인 분석

02 팀 구성 및 역할 TEAM MEMBER



기획
박시현

프로젝트 계획 수립



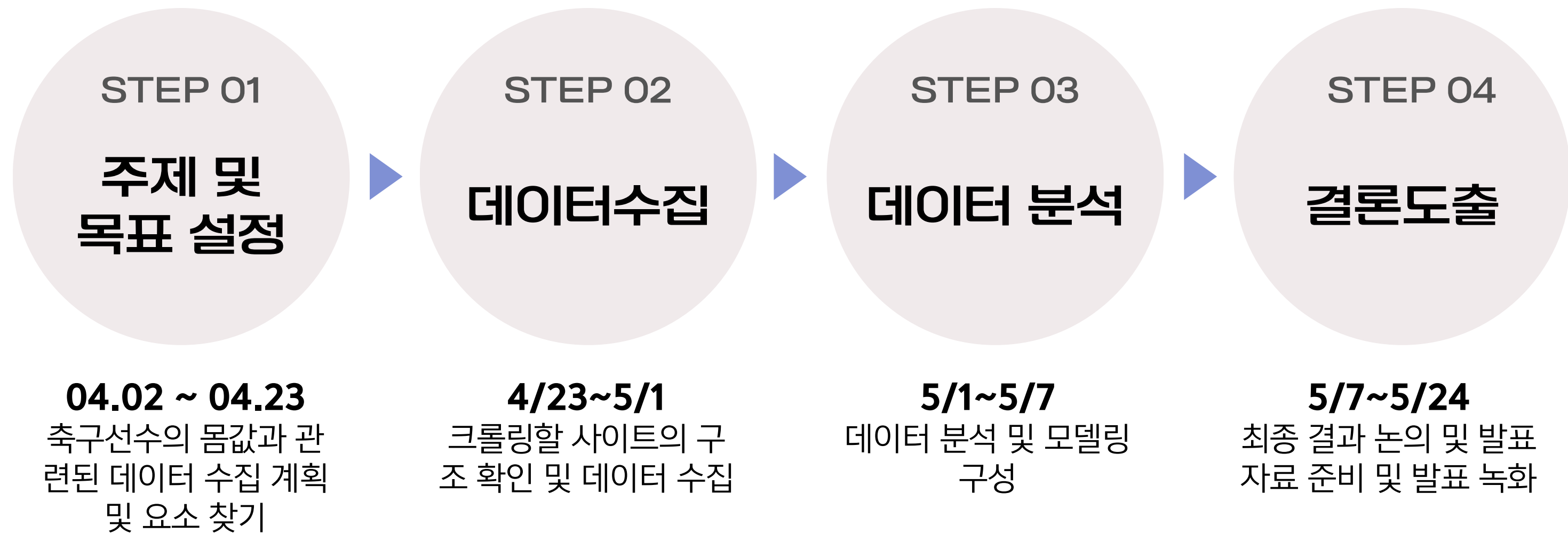
개발
원종우

데이터 수집, 분석

PPT는 공동제작

03 프로젝트 진행과정

WORK FLOW

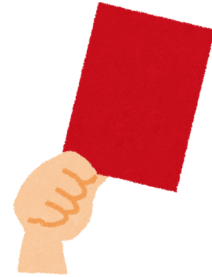
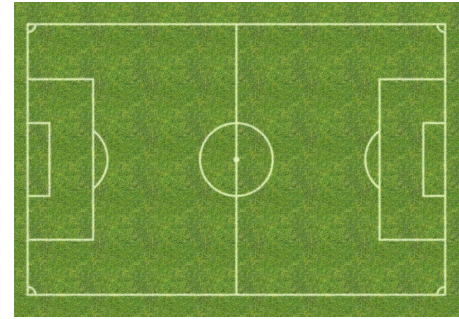


03 데이터 계획 수립



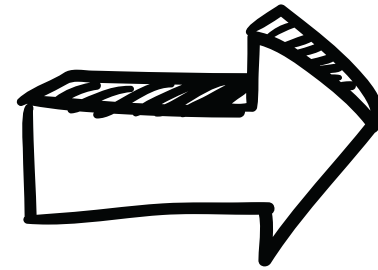
축구선수

어떤 포지션을
대상으로 할 것인가?
몇명으로 할 것인가?



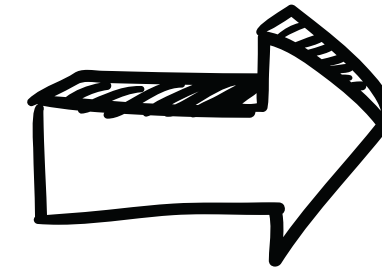
수집 할 데이터

요소들을 수집하기
위하여 어떤 것을
쓸 것인가.



모델링

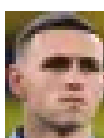
가져온 데이터를
통해 어떤 모델을
구성할 것인가.



예측 및 중요도 확인

모델링을 통한 모델의
성능 확인 및 요소들의
중요도 확인

03 데이터 선정 및 수집 과정

#	Player	Age	Nat.	Club	Market value	Matches								
1	 Erling Haaland Centre-Forward	24			€200.00m	51	39	0	5	2	0	0	1	11
2	 Vinicius Junior Left Winger	24	 		€200.00m	50	20	0	16	15	0	1	7	19
3	 Lamine Yamal Right Winger	17	 		€180.00m	59	19	0	26	4	0	0	4	28
4	 Kylian Mbappé Centre-Forward	26	 		€170.00m	58	40	0	4	5	0	1	5	19
5	 Bukayo Saka Right Winger	23	 		€150.00m	39	12	0	14	6	0	0	6	20
6	 Phil Foden Right Winger	24			€130.00m	48	10	0	7	2	0	0	15	18

[https://www.transfermarkt.com/spieler-statistik/wertvollstespieler/marktwertetop?
land_id=0&ausrichtung=Sturm&spielerposition_id=alle&altersklasse=alle&jahrgang=0&kontinent_id=0&plus=1](https://www.transfermarkt.com/spieler-statistik/wertvollstespieler/marktwertetop?land_id=0&ausrichtung=Sturm&spielerposition_id=alle&altersklasse=alle&jahrgang=0&kontinent_id=0&plus=1)

03 수집한 데이터

		국적	현재 시장가	골	옐로카드	교체	인	아웃						
#	Player	Age	Nat.	Club	Market value	Matches								
1	 Erling Haaland Centre-Forward	24			€200.00m	52	40	0	5	2	0	0	1	12
선수 이름		나이	소속팀		경기 수		어시스트			레드카드				

03 데이터 수집 과정

구글 시트

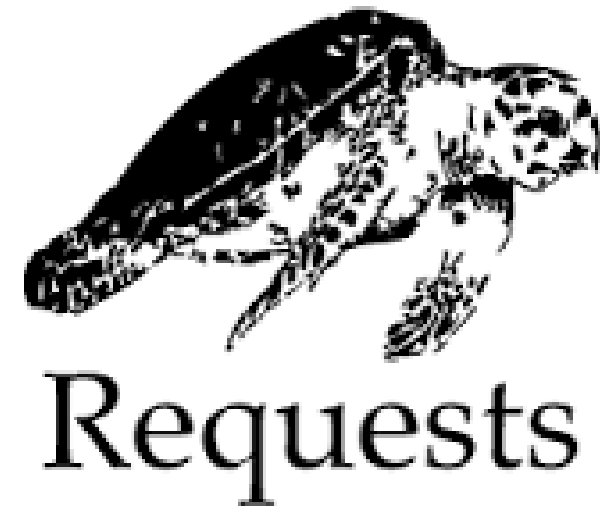
=IMPORTXML(A1,"//*[@id='yw1']/table/tbody/tr[1]/td[2]/table/tbody/tr[1]/td[2]/a")							
A	B	C	D	E	F	G	H
https://www.transfermarkt.com/spieler-statistik/wertvollstespieler/marktwertetop							
#N/A							

구글 시트로 진행 하였을 때는 일부 사이트에서 값이 #N/A로 뜨거나 제대로 들어 오지않는 것을 확인하였고 또한 500명의 데이터를 받는다고 하였을 때 어려움을 겪을 것으로 판단하여 파이썬의 request,beautifulsoup(크롤링)을 사용하여 수집을 진행 하기로 하였다.

03 사용한 기술들



BeautifulSoup



03 소스코드 설명 (수집)

```
for page in range(1, 25):
    print(f"Fetching page {page}...")
    url = base_url.format(page)
    res = requests.get(url, headers=headers)
    soup = BeautifulSoup(res.content, "lxml")

    rows = soup.select("table.items > tbody > tr")

    for row in rows:
        try:
            # 이름
            name_tag = row.select_one("td:nth-of-type(2) table tr td:nth-of-type(2) a")
            name = name_tag.text.strip() if name_tag else ""

            # 나이
            age = row.select_one("td:nth-of-type(3)").text.strip()

            # 국적 (img alt)
            nationality_tag = row.select_one("td:nth-of-type(4) img")
            nationality = nationality_tag["title"] if nationality_tag else ""
```

03 소스코드 설명 (수집)

```
for page in range(1, 25):  
    print(f"Fetching page {page}...")  
    url = base_url.format(page)  
    res = requests.get(url, headers=headers)  
    soup = BeautifulSoup(res.content, "lxml")
```



transfermarkt 링크 받기

```
rows = soup.select("table.items > tbody > tr")
```

```
for row in rows:
```

```
    try:
```

```
        # 이름
```

```
        name_tag = row.select_one("td:nth-of-type(2) table tr td:nth-of-type(2) a")
```

```
        name = name_tag.text.strip() if name_tag else ""
```

```
        # 나이
```

```
        age = row.select_one("td:nth-of-type(3)").text.strip()
```

```
        # 국적 (img alt)
```

```
        nationality_tag = row.select_one("td:nth-of-type(4) img")
```

```
        nationality = nationality_tag["title"] if nationality_tag else ""
```

03 소스코드 설명 (수집)

```
for page in range(1, 25):
    print(f"Fetching page {page}...")
    url = base_url.format(page)
    res = requests.get(url, headers=headers)
    soup = BeautifulSoup(res.content, "lxml")

    rows = soup.select("table.items > tbody > tr")
```

```
for row in rows:
    try:
        # 이름
        name_tag = row.select_one("td:nth-of-type(2) table t
        name = name_tag.text.strip() if name_tag else ""

        # 나이
        age = row.select_one("td:nth-of-type(3)").text.strip

        # 국적 (img alt)
        nationality_tag = row.select_one("td:nth-of-type(4)
        nationality = nationality_tag["title"] if nationalit
```

[illegible]

03 소스코드 설명 (수집)

```
for page in range(1, 25):
    print(f"Fetching page {page}...")
    url = base_url.format(page)
    res = requests.get(url, headers=headers)
    soup = BeautifulSoup(res.content, "lxml")

    rows = soup.select("table.items > tbody > tr")

    for row in rows:
        try:
            # 이름
            name_tag = row.select_one("td:nth-of-type(2) table tr td:nth-of-type(2) a")
            name = name_tag.text.strip() if name_tag else ""

            # 나이
            age = row.select_one("td:nth-of-type(3)").text.strip()

            # 국적 (img alt)
            nationality_tag = row.select_one("td:nth-of-type(4) img")
            nationality = nationality_tag["title"] if nationality_tag else ""
```

```
<table class="items">
  <thead>...</thead>
  <tbody>
    <tr class="odd">
      <td class="zentriert">1</td>
      <td class=
        <table class="inline-table">
          <tbody>
            <tr>
              <td rowspan="2">...</td>
              <td class="hauptlink">
                <a title="Erling Haaland" href=
                  _">Erling Haaland</a> == $0
              </td>
            </tr>
```

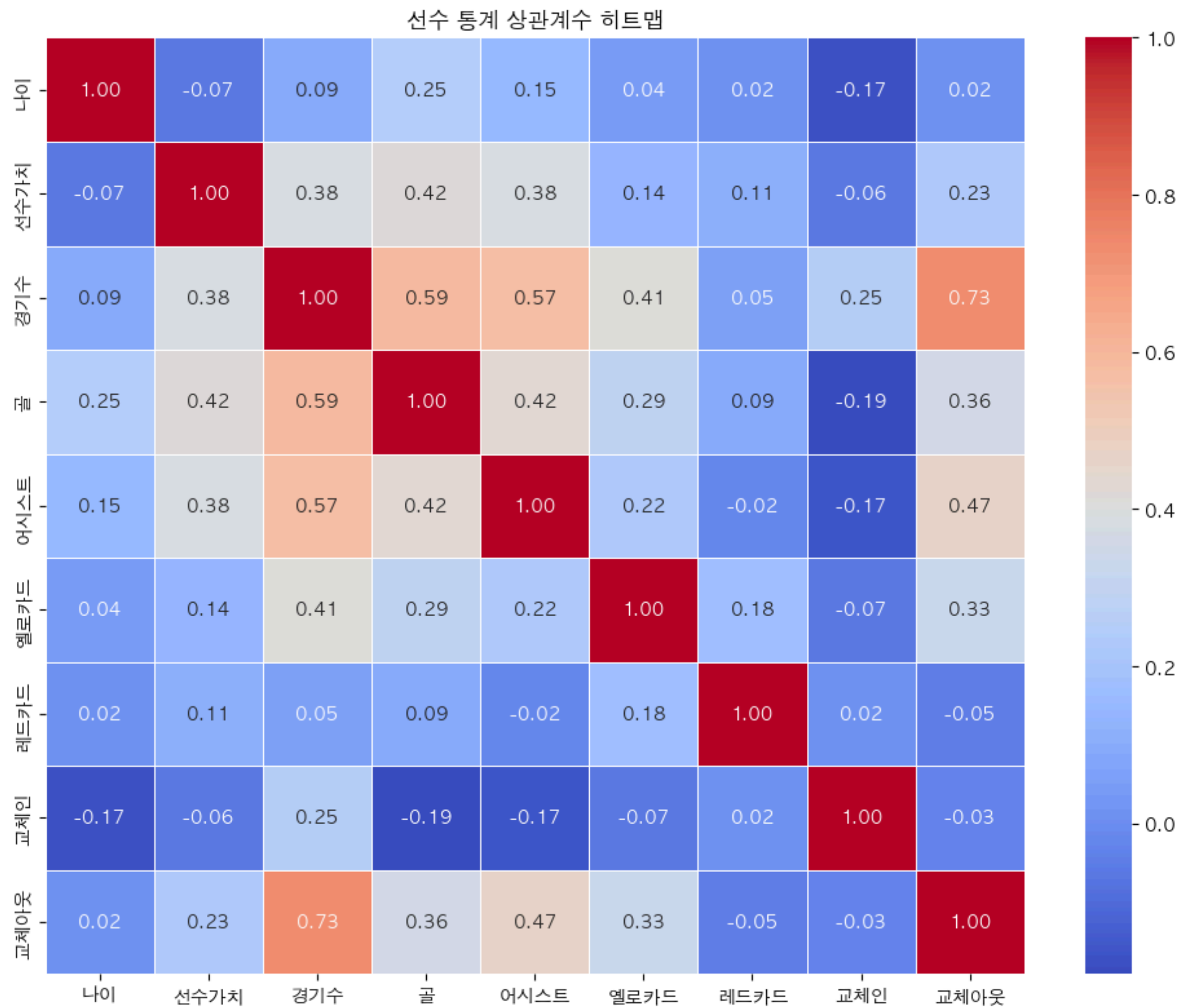
CSS 선택자를 사용하여 수집

03 데이터 수집 결과

1	이름, 나이, 국적, 소속클럽, 선수가치, 경기수, 골, 어시스트, 옐로카드, 레드카드, 교체인, 교체아웃	484	Cengiz Ünder, 27, Türkiye, Los An
2	Vinicius Junior, 24, Brazil, Real Madrid, €200.00m, 51, 20, 17, 15, 1, 8, 19	485	Myrto Uzuni, 30, Albania, Austin
3	Lamine Yamal, 17, Spain, FC Barcelona, €180.00m, 60, 19, 26, 4, 0, 4, 28	486	Milot Rashica, 28, Kosovo, Besikt
4	Erling Haaland, 24, Norway, Manchester City, €180.00m, 52, 40, 5, 2, 0, 1, 12	487	Fedor Chalov, 27, Russia, PAOK Th
5	Kylian Mbappé, 26, France, Real Madrid, €170.00m, 59, 42, 4, 5, 1, 5, 19	488	Michael Olunga, 31, Kenya, Al-Duh
6	Bukayo Saka, 23, England, Arsenal FC, €150.00m, 40, 12, 14, 6, 0, 7, 20	489	Cristian Arango, 30, Colombia, Sa
7	Alexander Isak, 25, Sweden, Newcastle United, €120.00m, 48, 32, 8, 1, 0, 1, 27	490	Lukas Haraslin, 29, Slovakia, AC
8	Rodrygo, 24, Brazil, Real Madrid, €100.00m, 50, 13, 10, 0, 0, 12, 28	491	Paulinho, 32, Portugal, Deportivo
9	Phil Foden, 25, England, Manchester City, €100.00m, 49, 10, 7, 2, 0, 16, 18	492	Vincenzo Grifo, 32, Italy, SC Fre
10	Lautaro Martínez, 27, Argentina, Inter Milan, €95.00m, 49, 22, 7, 3, 0, 5, 25	493	Anthony Martial, 29, France, AEK
11	Julián Álvarez, 25, Argentina, Atlético de Madrid, €90.00m, 54, 29, 7, 6, 0, 10, 27	494	Marvin Ducksch, 31, Germany, SV W
12	Harry Kane, 31, England, Bayern Munich, €90.00m, 53, 43, 13, 7, 0, 5, 19	495	Thiago Fernández, 21, Argentina,
13	Michael Olise, 23, France, Bayern Munich, €80.00m, 56, 18, 22, 4, 0, 10, 28	496	Keny Arroyo, 19, Ecuador, Besikta
14	Khvicha Kvaratskhelia, 24, Georgia, Paris Saint-Germain, €80.00m, 51, 14, 11, 4, 0, 7, 24	497	Rômulo, 23, Brazil, Göztepe, €7.00
15	Raphinha, 28, Brazil, FC Barcelona, €80.00m, 57, 34, 25, 7, 0, 4, 16	498	Rafiu Durosini, 22, Nigeria, FC
16	Omar Marmoush, 26, Egypt, Manchester City, €75.00m, 52, 29, 17, 3, 0, 6, 24	499	Max Moerstedt, 19, Germany, TSG 1
17	Rafael Leão, 25, Portugal, AC Milan, €75.00m, 58, 13, 15, 7, 0, 13, 27	500	Sindre Walle Egeli, 18, Norway, F
18	Viktor Gyökeres, 26, Sweden, Sporting CP, €75.00m, 58, 63, 17, 6, 0, 8, 5	501	John Kennedy, 23, Brazil, CF Pach
19	Marcus Thuram, 27, France, Inter Milan, €75.00m, 53, 18, 9, 3, 0, 11, 23		

총 500명의 이름, 나이, 국적, 소속클럽, 선수 가치, 경기 수, 골, 어시스트, 옐로카드, 레드카드, 교체인, 교체아웃 데이터를 수집하였다

03 데이터 수집 결과



03 데이터 수집 결과 (분석)

결측치

```
print(df.isnull().sum())
```

이름	0
나이	0
국적	0
소속클럽	0
선수가치	0
경기수	0
골	0
어시스트	0
옐로카드	0
레드카드	0
교체인	0
교체아웃	0
dtype:	int64

결측치 확인

정보

```
print(df.info())
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   이름        500 non-null    object
1   나이        500 non-null    int64
2   국적        500 non-null    object
3   소속클럽    500 non-null    object
4   선수가치    500 non-null    object
5   경기수      500 non-null    int64
6   골          500 non-null    int64
7   어시스트    500 non-null    int64
8   옐로카드    500 non-null    int64
9   레드카드    500 non-null    int64
10  교체인      500 non-null    int64
11  교체아웃    500 non-null    int64
dtypes: int64(8), object(4)
memory usage: 47.0+ KB
None
```

데이터 정보

유일값 확인

```
print(df.nunique())
```

✓ 0.0s

이름	498
나이	21
국적	74
소속클럽	169
선수가치	46
경기수	62
골	44
어시스트	24
옐로카드	15
레드카드	3
교체인	35
교체아웃	38
dtype:	int64

유일값 확인

03 데이터 수집 결과 (분석)

```
print(df['이름'].value_counts())  
✓ 0.0s  
  
이름  
Paulinho      2  
Pedro         2  
Vinicius Junior 1  
Mauro Icardi  1  
Roland Sallai 1  
..  
Pepê         1  
Amine Adli   1  
Lorenzo Lucca 1  
Beto         1  
John Kennedy  1  
Name: count, Length: 498, dtype: int64
```

Paulinho와 Pedro라는 이름의 선수가
각각 중복 값 이었다

03 소스코드 설명 (랜덤 포레스트 모델링)

```
df['선수가치'] = df['선수가치'].str.replace('€', '').str.replace('m', '').astype(float)
df.head()
```

	이름	나이	국적	소속클럽	선수가치	경기수	골	어시스트	옐로카드	레드카드	교체인	교체아웃
0	Vinicius Junior	24	Brazil	Real Madrid	200.0	51	20	17	15	1	8	19
1	Lamine Yamal	17	Spain	FC Barcelona	180.0	60	19	26	4	0	4	28
2	Erling Haaland	24	Norway	Manchester City	180.0	52	40	5	2	0	1	12
3	Kylian Mbappé	26	France	Real Madrid	170.0	59	42	4	5	1	5	19
4	Bukayo Saka	23	England	Arsenal FC	150.0	40	12	14	6	0	7	20

```
df.drop(columns=['이름'], inplace=True)
```

```
df = pd.get_dummies(df, columns=['국적', '소속클럽'])
```

```
X = df.drop(columns=['선수가치'])
y = df['선수가치']
```

데이터 정제 및 인코딩

- 선수 가치(€m)를 float형으로 변환
- 범주형 변수(국적, 소속클럽)는 원-핫 인코딩
- 예측에 불필요한 '이름' 컬럼 제거

03 소스코드 설명 (랜덤 포레스트 모델링)

```
X = df.drop(columns=['선수가치'])  
y = df['선수가치']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = RandomForestRegressor(n_estimators=100, random_state=42)  
model.fit(X_train, y_train)
```

```
▼ RandomForestRegressor  
RandomForestRegressor(random_state=42)
```

```
y_pred = model.predict(X_test)
```

회귀 모델 구성

- 입력 변수(X): 국적, 소속 클럽 등
- 목표값(y): 선수 가치 (float)
- 데이터 분할: 학습 80% / 테스트 20%
- 사용 모델:
RandomForestRegressor(n_estimators=100)

03 소스코드 설명 (랜덤 포레스트 모델링)

```
y_pred = model.predict(X_test)
```

```
print("✅ MSE (평균제곱오차):", mean_squared_error(y_test, y_pred))
```

```
print("✅ R^2 Score (결정계수):", r2_score(y_test, y_pred))
```

✓ 0.0s

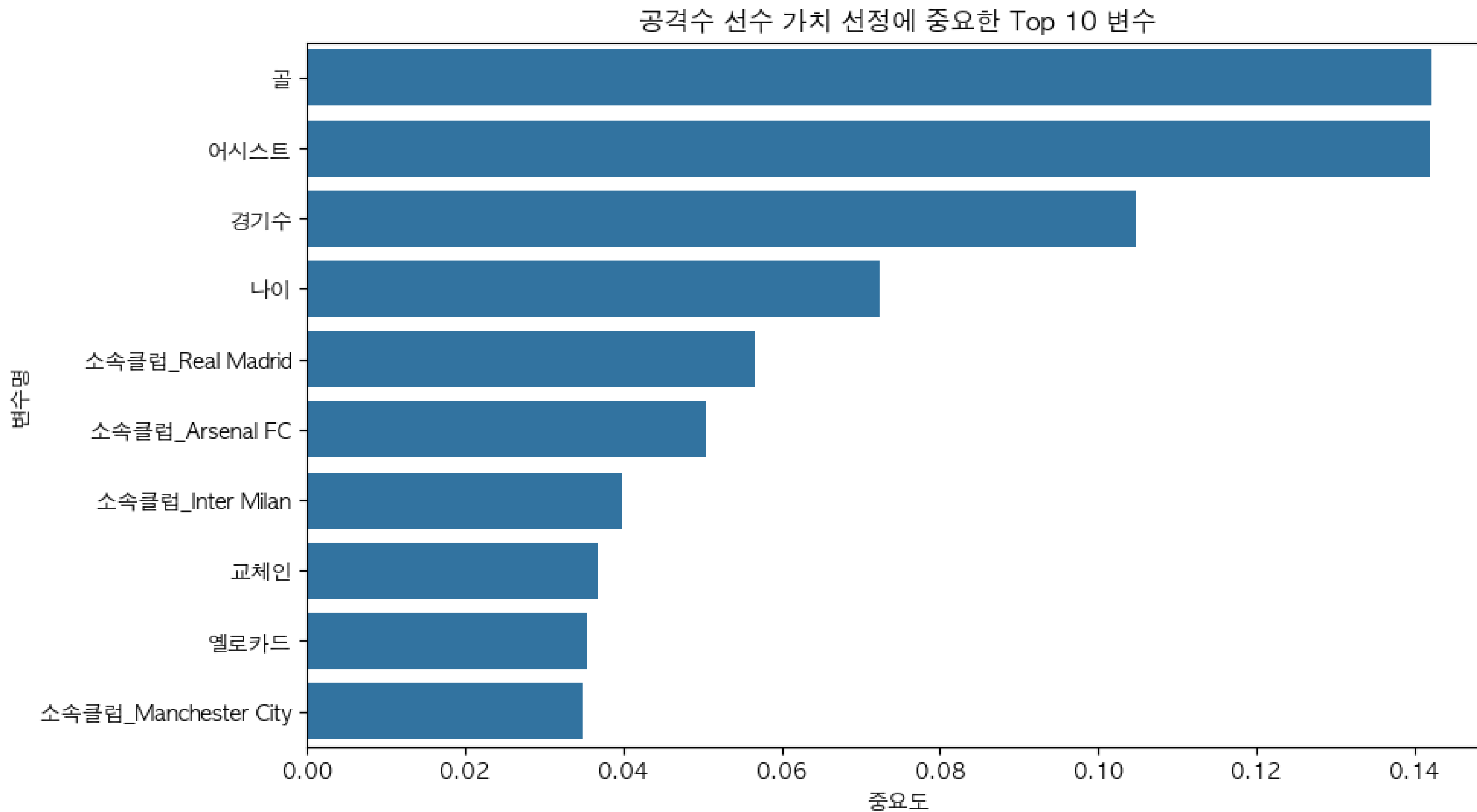
✅ MSE (평균제곱오차): 492.756033

✅ R^2 Score (결정계수): 0.443304040167932

회귀 모델 구성모델 성능 평가 결과

- 평균제곱오차 (MSE): 예측과 실제값 차이의 제곱 평균
- 결정계수 (R^2 Score): 1에 가까울수록 좋은 성능
- 예측값과 실제값의 관계 시각화 가능

03 데이터 분석 결과



04 평가 (어려웠던 점) 데이터 수집 과정의 난관

- 문제 1

다양한 소스 통합의 어려움

- 문제 2

API 제한으로 인한 지연 → `time.sleep()`를 이용하여 해결

- 문제 3

비정형 데이터 처리 복잡성 → BeautifulSoup 활용

04 평가 (어려웠던 점) 팀 협업에서의 도전 과제

- 문제 1

의사소통 문제로 인한 중복 작업 → 더 나은 소스를 활용

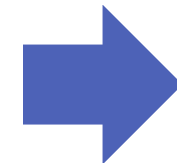
- 문제 2

원격 협업 툴 적응 문제

- 문제 3

역할 분담의 불균형 → 대면 회의 후 해결

04 성과 및 결론도출 CONCLUSIONS



처음에는 Selenium과 WebDriver를 활용한 데이터 수집 과정에서 문제를 겪었으나 BeautifulSoup로 바꾼 뒤 모든 것들이 다 순조롭게 해결 되었다. 모델 정확도를 높이는 작업을 한다면 보다 합리적인 의사결정을 내리는 모델이 될 것이다.

THANK YOU

감사합니다

프로젝트에 사용된 소스코드와 데이터는

<https://github.com/Jongwoo0101/DataCOLndCLE> 에서 확인하실 수 있습니다